Module 1: Java

Java OOP

It's objects all the way down.

0.

Forewarning

…but everything changed when inheritance attacked.

“Object-oriented programming is an exceptionally bad idea which could only have originated in California.”

~ Edsger Dijkstra

uinverse

FizzBuzzEnterpriseEdition / src / main / java / com / seriouscompany / business / java / fizzbuzz / packagenamingpackage / impl / math / arithmetics / NumberIsMultipleOfAnotherNumberVerifier.java

Top

Code Blame 55 lines (45 loc) · 1.69 KB Raw

```
16  public class NumberIsMultipleOfAnotherNumberVerifier {
36          * @return
37          */
38         public static boolean numberIsMultipleOfAnotherNumber(final int nFirstNumber, final int nSecondNumber)
39                 try {
40                         final int nDivideFirstIntegerBySecondIntegerResult =
41                                         (NumberIsMultipleOfAnotherNumberVerifier.integerDivider.divide(nFirstNu
42                         final int nMultiplyDivisionResultBySecondIntegerResult =
43                                         nDivideFirstIntegerBySecondIntegerResult * nSecondNumber;
44                         if (IntegerForEqualityComparator.areTwoIntegersEqual(nMultiplyDivisionResultBySecondInt
45                                         nFirstNumber)) {
46                                 return true;
47                         } else {
48                                 return false;
49                         }
50                 } catch (final ArithmeticException ae) {
51                         return false;
52                 }
53         }
54
```

All Symbols

field integerDivider

Definition Search

In this file

```
18 IntegerDivider integerDivider;
```

2 References Search

In this file

```
30 this.integerDivider = applicationContext.g
41 integerDivider.divide(nFirstNumber, nSecon
```

Search for this symbol

FizzBuzzEnterpriseEdition (GitHub)

```
PS C:\dev\FizzBuzzEnterpriseEdition> cloc src
      89 text files.
      89 unique files.
       0 files ignored.

github.com/AlDanial/cloc v 1.96  T=0.17 s (532.6 files/s, 15686.2 lines/s)
-------------------------------------------------------------------------------
Language                     files          blank        comment           code
-------------------------------------------------------------------------------
Java                            89            496            739           1386
-------------------------------------------------------------------------------
SUM:                            89            496            739           1386
-------------------------------------------------------------------------------
PS C:\dev\FizzBuzzEnterpriseEdition> 1386 / 89
15.5730337078652
```

FizzBuzz cloc (Count Lines of Code)

1.

Defining OOP

There are two hard problems in computer science…

Possible Definitions

1. The "Book" Definitions

Object-oriented programming is typically defined in terms of either:

- Classes & objects, along with some idea of methods/inheritance.
- A more advanced idea is Alan Kay's messaging passing; focused on encapsulation and polymorphism.
  - Less modern OOP, more Actors

2. The "Idea" Definitions

An object-oriented language typically supports the following functionality:

- Allows encapsulation to combine data/state (fields) and supported operations (methods) in one container.
- Has some form of polymorphism, commonly as a hierarchy of types.
- Typically allows some concept of inheritance (aka, behavior dependent on parent classes/interfaces).

2.

Classes

This is where the fun begins!

```java
public class Vector {                          // class: object “blueprint”

  private int x; private int y;                // fields: data stored on objects

  public Vector(int x, int y) {                // constructors: special functions
    this.x = x; this.y = y;                    // used to initialize new objects
  }                                            // this: the current object

  public double getLength() {                  // methods: functions on objects
    return Math.sqrt(x * x + y * y);           // (meaningful operations), enables
  }                                            // inheritance and polymorphism

}

Vector vec = new Vector(3, 4);        // new: create an instance (object)
assert vec.getLength() == 5.0;        // call/invoke methods
```

Java Classes: Fields, Constructors, & Methods

```java
public class Vector {

  private int x;                                  // fields are often kept
  private int y;                                  // private for encapsulation

  public int getX() { return x; }                 // getter: field x
  public void setX(int x) { this.x = x; }         // setter: field x

  public int getY() { return y; }                 // getter: field y
                                                  // no setter!
}

Vector vec = new Vector(3, 4);
int x = vec.getX();     vec.setX(2 * x);     // x getter/setter
int y = vec.getY();     vec.setY(2 * y);     // y getter (no setter!)
```

Java Classes: Field Getters/Setters

```java
public class Vector {
  @Override //recommended                          // @Override: defined by parent
  public boolean equals(Object other) {            // equals: checks if objects
    return other instanceof Vector                 // are logically equal
      && x == ((Vector) other).x                   // `==`: reference equality
      && y == ((Vector) other).y;                  // (but works for primitives)
  }

  public int hashCode() {                          // hashCode: returns an int
    return Objects.hash(x, y);                 // key used in HashMap lookups
  }                                                // equality: equals → hashCode

  public String toString() {                       // toString: returns a String
    return "<" + x + ", " + y + ">";               // representation of the object
  }                                                // (called when printing)
}
```

Java Classes: *equals*, *hashCode*, and *toString*

State Invariants

State

The **state** of an object is the current value of any stored data (fields).

- Typically use “state” when referring to mutable objects/data.
- Classes control state for objects
  - Encapsulation: private state
  - Inheritance: inherited classes can still have private state
- “Global” state for the environment

Invariant

An **invariant** is a condition about the state of an object which defines “valid” state.

- Both **precondition** & **postcondition**
- Invariants are “always true”, but may be temporarily broken within methods
  - e.g. updating two fields
  - Common during initialization
- Invariants also apply to inheritance, but that’s for another time (or class).

```java
public class UnitVector {

  private double x, y;                          // invariant: x² + y² == 1

  public UnitVector(Vector v) {                 // constructor: establishes
    x = v.getX() / v.getLength();               // state invariants (fields)
    y = v.getY() / v.getLength();
  }

  public void rotate(double radians) {          // method: responsible for
    x = …; //breaks invariant!                  // maintaining invariants
    y = …; //restores invariant!
  }

}
```

State Invariants:

What if a class doesn’t actually need invariants?

Can we benefit from restricting this?

3.

Records

or structs, or data classes, or object literals, or-

Java Records

“Immutable POJO’s”

Records act like simplified POJO’s (Plain ‘Old Java Objects).

- Records define “simple” objects focused on data access/storage
  - Fields & getters (no setters!)
  - equals, hashCode, & toString
- Records are immutable!
  - POJO’s *can* be mutable, but this is rare (& mutable state is hard!)

Copy Invariant

Formalizes the idea of “no internal state”: records are simply “carriers” of field values.

- Given a record R and instance r:
  - `copy = new R(r.c1(), r.c2(), …)`
- Then:
  - `r.equals(copy)`
- Note that since records are Objects, the identity of these objects will be different:
  - `r != copy`

```java
public final class Vector {
  private final int x;
  private final int y;

  public Vector(int x, int y) { … }

  // “fluent” getters: x() = getX()
  public int x() { … }
  public int y() { … }

  public boolean equals(Object other) { … }
  public int hashCode() { … }
  public String toString() { … }
}
```

```java
public record Vector(
  int x,
  int y
) {}
```

Java Classes vs Records

Module 1: Java

Java OOP

It's objects all the way down.

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals